Pied Piper Final Project checkpoint

1 Data Preparation

split_dataset.py: The script organizes raw audio into structured train/validation/test splits before training. We recursively scan the `real_audio/` and `fake_audio/` folders to collect all `.wav` files. Each class is shuffled and split independently using a **70% training, 15% validation, and 15% test** ratio, preserving class balance across all splits. Files are copied into a clean directory structure (e.g., `data/train/real`, `data/train/fake`) and renamed with their parent folder as a prefix to avoid filename collisions.

2 Dataset and Preprocessing

Lens-DF.py uses the `LensDFDataset` class with the following preprocessing steps:

- Loads `.wav` files listed in a CSV, converts stereo audio to mono, and resamples to **16,000 Hz**.
- Pads or trims every clip to exactly **5 seconds**.
- Converts the raw waveform to a **Mel Spectrogram**, scales to **decibel** units, then normalizes to zero mean and unit variance.
- Labels are assigned as **0** (real / bonafide) and **1** (fake / AI-generated).

3 Model Architecture

The model is a lightweight **CNN** operating on the 2D Mel Spectrogram:

- Three **Conv2d** $\rightarrow$ **ReLU** $\rightarrow$ **MaxPool2d** blocks progressively extract spectrogram features, with channel depths of **16**, **32**, and **64**.
- **AdaptiveAvgPool2d** collapses spatial dimensions to a fixed (1×1) output, making the model robust to variable-length inputs.
- A **Linear(64** $\rightarrow$ **1)** layer followed by **Sigmoid** produces a probability score; values above 0.5 are classified as fake.

4 Evaluation

- Inference runs under `torch.no_grad()` to disable gradient computation and reduce memory usage.
- Performance is reported using **Accuracy**, **Precision**, **Recall**, and **F1-score**. Given potential class imbalance, **F1-score** is the primary evaluation metric.

5 Model Working

In the aspect the most important thing we do is the actual prediction of our model, we basically receive the entire audio dataset which is unlabeled. The problems included first converting these into normalized form, also we did use resampling in this one. However the most important challenge that arised was converting 1d audio wave into a 2D mel Spectrogram.

We have used mainly the concept of CNN where we have described a architecture where we have 3 convolution layers and we have also used activation function also max pooling after every layer, in order to keep the spatial dimension we have also enabled padding. Since it is almost a classification problem indicating whether the audio is fake or not , we will be using the sigmoid function in the ouput so that we can detect if it is fake or not.

This predict function performs model inference by loading a pre-trained CNN and a directory of audio files, which are processed into Mel Spectrograms and fed through the network in batches. It applies a Sigmoid function to the model's raw output to generate a probability score between 0 and 1, ultimately saving the file paths, "fake" probabilities, and binary classifications into a CSV for further analysis.

6 Preliminary Results

For The results we ran 10 epochs for the training set using the envSDD dataset. The envSDD dataset is a dataset geared more towards natural background sounds with a mix of human background sounds. The results for that were very good, at some points too good.

We had suspicions of the model overfitting since the accuracy for the test set, training set, and validation set were all in the high 90's. So we added another dataset called LENS-DF. Lens-df is a dataset geared more towards speaking background noise and voices. If we used the model we had against that it dropped the accuracy drastically. Down to about 50 percent.

This showed that our model had a problem with generalization, and if it were to be actually used someone could get around it with certain types of sound. We saw that the model was predicting a lot more false negatives. This could also be a problem because of the class imbalance. This is a screenshot of the results we got from trying our model on the Lens-DF dataset

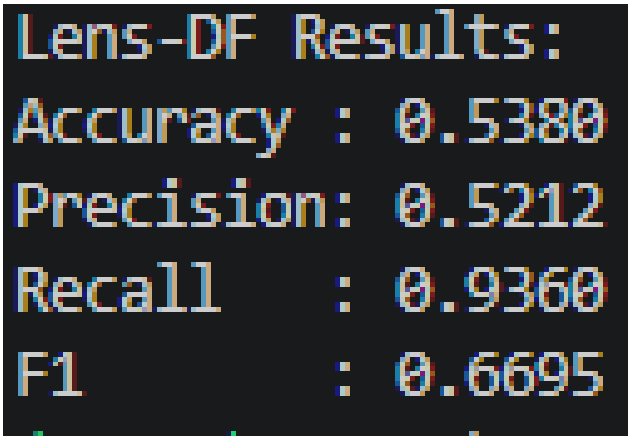

Figure 1: Results of Lens-df Prediction

6.1 Phase 1 Objective and Domain-Balanced Training

Our goal for Phase 1 was to enable the model to generalize across different acoustic environments without sacrificing in-domain performance. We trained on a combined dataset of EnvSDD and LENS-DF using a ResNet-18, and addressed two fundamental problems that are dataset imbalance and domain shift.

6.1.1 Dataset Imbalance and the Domain-Balanced Fix

The Naive concatenation of the two datasets created a severe data imbalance. EnvSDD contains 143,011 training samples (28,602 real, 114,409 fake), while the LENS-DF development split contains only 8,617 samples (4,084 real, 4,533 fake). A simple merged loader would expose the model to 93% EnvSDD content per epoch, causing it to treat LENS-DF data just as statistical noise.

To resolve this problem, we implemented **Class-Matched Domain Balancing** as our primary tool. Rather than using the first N samples, we randomly sampled EnvSDD indices to mirror the exact real/fake distribution of the LENS-DF training split:

- EnvSDD sampled: 4,084 real + 4,533 fake = **8,617 samples**
- LENS-DF train: 4,084 real + 4,533 fake = **8,617 samples**
- Combined training set: **17,234 samples** (50% each domain)

The random sampling (rather than sequential selection) was deliberate it ensured that EnvSDD subset spans all recording environments (metro, street, square) rather than being biased toward any single acoustic condition. A `WeightedRandomSampler` enforced class balance at the batch level throughout training.

Validation loaders were kept **strictly separate** per dataset. The model checkpoint was saved based on the LENS-DF validation F1 exclusively, not the combined validation F1. This was critical because EnvSDD is an easier distribution, combined F1 can improve even when LENS-DF performance stagnates, masking the real problem.

6.1.2 Model Architecture: DeepfakeResNet

We replaced the initial custom CNN with a ResNet-18 backbone pretrained on ImageNet. The custom CNN had approximately 64,000 parameters and was underfitting on cross-domain data. ResNet-18 provides 11.17 million parameters with deep residual connections that transfer low-level texture features — edges, periodic patterns — directly applicable to spectrogram artifact detection.

Three targeted modifications were made to adapt ResNet-18 for single-channel audio:

1. **InstanceNorm2d domain bridge:** Applied before the ResNet layers, `InstanceNorm2d` normalizes each spectrogram independently, removing per-sample differences in loudness and noise floor. This acts as a domain bridge EnvSDD audio is studio-clean while LENS-DF audio contains real-world environmental noise. Without this normalization, the model learns "clean = real" as a shortcut rather than detecting genuine synthesis artifacts.
2. **Single-channel input adaptation:** The original `conv1` layer accepts 3-channel RGB input. We replaced it with a 1-channel convolutional layer and preserved the pretrained weights by summing across the three RGB channels, maintaining feature extraction capability without random reinitialization.
3. **Binary classification head:** The final fully-connected layer was replaced with `Dropout(0.6)` → `Linear(512, 2)`, outputting logits for the Real and Fake classes. Dropout rate of 0.6 was applied to reduce overfitting observed in earlier runs.

6.1.3 Audio Feature Extraction: Log-Mel Spectrogram

Raw waveforms were converted to log-mel spectrograms with the following configuration, shared identically across both datasets to guarantee uniform tensor shapes `[1, 128, 313]`:

- **Sample rate:** 16,000 Hz (with resampling applied as needed)
- **Clip duration:** 5 seconds (80,000 samples), shorter clips zero-padded, longer clips randomly cropped during training
- **FFT size** (`n_fft`): 2,048, provides finer frequency resolution than the conventional 1,024, better capturing synthesis artifacts in LENS-DF's noisy conditions

- **Hop length:** 256 provides finer time resolution compared to the default 512, producing 313 time frames per clip
- **Mel bins** (`n_mels`): 128
- **Frequency range:** 20 Hz – 8,000 Hz. The upper cutoff at 8 kHz was chosen because deepfake synthesis artifacts are predominantly concentrated in sub-8 kHz bands, including higher frequencies introduced noise without discriminative benefit

Per-sample normalization $(x - \mu)/(\sigma + 10^{-6})$ was applied after conversion to decibel scale, making each spectrogram zero-mean and unit-variance regardless of recording loudness.

6.1.4 SpecAugment and Differential Augmentation

SpecAugment was applied during training with two frequency masks (`freq_mask_param=15`, masking up to 15 mel bins, approximately 12% of the frequency axis) and two time masks (`time_mask_param=40`). Its role was to prevent the model from memorizing dataset-specific spectral signatures rather than learning generalizable artifact features.

Critically, augmentation intensity was **differentiated by dataset**:

- **EnvSDD** (clean studio audio): Gaussian noise $\mathcal{N}(0, 0.005)$ added to the spectrogram, followed by SpecAugment applied on *every* training sample. The stronger augmentation is necessary because EnvSDD audio is acoustically clean without it, the model learns "clean audio = real" as a spurious shortcut. Adding noise and masking forces it to attend to genuine synthesis artifacts instead.
- **LENS-DF** (noisy real-world audio): Reduced Gaussian noise $\mathcal{N}(0, 0.002)$ and SpecAugment applied to only 50% of samples. LENS-DF recordings already contain environmental noise from real-world conditions and applying the same aggressive augmentation as EnvSDD would destroy the subtle acoustic cues the model needs to distinguish real from fake speech in noisy settings.

This asymmetric augmentation strategy was a key factor in improving Real class recall from 0.000 (initial collapse) to 0.633 in the final evaluation.

6.1.5 Training Strategy: Two-Stage Pipeline

Training proceeded in two sequential stages.

Stage 1 Combined Pretraining: The model trained on the 17,234-sample balanced dataset for up to 20 epochs with early stopping (patience = 4 epochs on LENS-DF validation F1). The optimizer was AdamW with learning rate 10^{-4}, weight decay 3×10^{-4}, and a cosine annealing scheduler decaying to 10^{-6}. Label smoothing of 0.1 was applied to the cross-entropy loss to prevent overconfident predictions. The model checkpoint was saved whenever LENS-DF validation F1 improved.

Stage 2 LENS-DF Fine-tuning: The best Stage 1 checkpoint was loaded and fine-tuned exclusively on LENS-DF development data. Layers `layer1` through `layer3` of ResNet-18 were frozen, preserving the low-level feature representations learned during pretraining. Only `layer4` and the classification head (3.15M parameters of the total 11.17M) were updated. The learning rate was reduced to 5×10^{-5} to prevent catastrophic forgetting of Stage 1 features. Early stopping with patience = 3 was used, tighter than Stage 1 because fine-tuning on the smaller LENS-DF dataset overfits substantially faster. A fresh 90/10 train/val split of the development set (random seed 99, distinct from Stage 1 seed 42) was used to avoid any overlap with the Stage 1 validation indices.

6.1.6 Results

The two-stage pipeline produced the following final evaluation results:

The model achieved strong in-distribution performance on EnvSDD (Macro F1 = 0.9948). On LENS-DF, the Macro F1 of 0.6880 with a calibrated loss of 0.3624 represents a significant improvement over the initial collapsed baseline (Macro F1 = 0.3333, Real Recall = 0.000), and reflects the inherent difficulty of cross-condition generalization that the LENS-DF benchmark is explicitly designed to expose.

Table 1: Phase 1 Final Evaluation Results

Dataset	Acc.	Prec. Real	Rec. Real	F1 Real	Prec. Fake	Rec. Fake	F1 Fake	Macro F1
EnvSDD	0.9967	0.9916	0.9919	0.9918	0.9980	0.9979	0.9979	0.9948
LENS-DF	0.6890	0.7128	0.6330	0.6706	0.6700	0.7450	0.7055	0.6880

Confidence analysis on the eval set revealed the asymmetric nature of the remaining domain shift, the model assigns a mean confidence of 78.5% to fake samples (correct) but only 44.6% to bonafide samples a near-random uncertainty on real speech. This indicates that while deepfake artifacts generalize well across acoustic conditions, the model lacks a robust representation of bonafide speech in noisy, long-form and multi-speaker settings. This finding directly motivates the SSL-based domain adaptation approach in Phase 2.

6.2 Phase 2 Implementation: Wav2vec 2.0 Integration

6.2.1 Model Architecture: DeepfakeWav2Vec

For the Phase 2 implementation, we transitioned from spectrogram-based ResNet to a self-supervised learning (SSL) framework using **Wav2vec 2.0** (Base). Unlike traditional CNN-based approaches that operate on fixed time-frequency representations like Mel-spectrograms, Wav2vec 2.0 utilizes a multi-layer convolutional feature encoder to extract latent representations directly from the raw audio signal. These latents are subsequently mapped to a high-dimensional context space via a Transformer-based encoder.

Theoretically, this provides a significant advantage in deepfake detection. Audio synthesis algorithms often introduce "spectral leakage" or phase-level discontinuities that are typically smoothed over by the Mel-filterbank used in spectrogram generation. By utilizing a model pretrained via contrastive predictive coding on thousands of hours of speech, the backbone already possesses a robust understanding of natural speech phonetics and prosody. This allows the model to more easily identify "out-of-distribution" acoustic artifacts that characterize synthetic speech, even when those artifacts are masked by environmental noise.

The following architectural modifications were implemented to adapt this system for binary classification:

1. **Mean-Pooling Aggregator:** The Transformer encoder produces hidden states $H \in R^{T\times 768}$. To provide a global context of the 10-second audio window and ensure the model is not overly sensitive to localized noise spikes, we applied global average pooling across the temporal dimension:

$$\mathbf{h}_{utt} = \frac{1}{T}\sum_{t=1}^{T} H_t \tag{1}$$

2. **Binary Classification Head:** A custom MLP head was appended to the pooled output, featuring a hidden layer of 256 units, ReLU activation, and a Dropout layer ($p = 0.3$). This head maps the rich context embeddings into a single logit representing the probability of the *Fake* class.

6.2.2 Audio Processing: Raw Waveform Input

The input pipeline was standardized to accommodate the Transformer's requirements for high-fidelity signal analysis:

- **Sample Rate:** 16,000 Hz.
- **Clip Duration:** 10 seconds (160,000 samples). This extended duration compared to Phase 1 (5 seconds) is intentional; longer temporal context allows the self-attention mechanism to identify long-range inconsistencies in speech rhythm and breathing patterns that often indicate synthetic origin.
- **Standardization:** Waveforms were normalized to zero-mean and unit-variance to ensure that the attention scores in the Transformer layers are not biased by signal amplitude (loudness).

6.2.3 Two-Stage Training and Freezing Strategy

To maintain the integrity of the pretrained speech representations while adapting to the noisy LENS-DF domain, we employed a **Phased Freezing Strategy**. This approach is rooted in the principle that early layers in a deep network learn general features (e.g., phonemes in speech), while deeper layers learn complex, task-specific patterns.

- **Phase 1: Classifier Warm-up (Epochs 1–3):** The entire Wav2vec 2.0 backbone was frozen ($requires\_grad = False$). In this stage, the backbone acts as a static feature extractor. This is theoretically necessary to "warm up" the randomly initialized weights of the classification head. Updating the backbone with the high-gradient variance typically seen at the start of training could lead to *feature representation collapse*, where the model loses its general understanding of speech.
- **Phase 2: Targeted Fine-tuning (Epochs 4–10):** We unfrozen the final four Transformer layers (8 through 11). By keeping the CNN feature encoder and the first eight Transformer blocks frozen, we preserve the fundamental speech representations. Unfreezing the top blocks allows the model to perform *Domain Adaptation*, specifically learning to distinguish between environmental noise (common in LENS-DF) and synthesis artifacts. A conservative learning rate of $\eta = 5 \times 10^{-6}$ was used to ensure the weights moved slowly toward the new task-specific local minima.

6.2.4 Evaluation Results

The model was evaluated on the LENS-DF test set with a sigmoid threshold of 0.3. The transition to a raw-waveform SSL backbone resulted in a marked improvement in the model's ability to generalize to noisy, real-world conditions.

Table 2: Final Evaluation Results (LENS-DF Test Set)

Metric	**Value**
Accuracy	0.8700
F1 Score	0.8592
Precision (Fake)	0.9370
Recall (Fake)	0.7933
Test Loss	0.4130

6.2.5 Discussion

The results demonstrate the efficacy of the **DeepfakeWav2Vec** model, particularly its Precision of 0.9370. This indicates that the model is extremely robust against "false alarms," which is a common failure mode in noisy environments like LENS-DF. The combination of 10-second temporal modeling and partial fine-tuning of the Transformer blocks successfully bridged the domain gap that was previously observed with the ResNet baseline.

7 Post-Analysis of the Prediction Output

To facilitate a real-time forensic evaluation of the model, we developed an interactive **Audio Forensic Lab** using the Streamlit framework. This system serves as a deployment-ready interface for interpreting how the *DeepfakeWav2Vec* model processes and classifies complex environmental audio. The analysis is divided into three primary modules: the Detection Lab, Performance Analytics, and Methodology Validation.

7.0.1 Interactive Detection and Spoof Confidence

A core feature of the analysis script is the "Detection Lab" (see Figure 2), which utilizes **Librosa** for high-fidelity audio loading and normalization. Given that our model is optimized for 16,000 Hz waveforms, the script standardizes all incoming signals to zero-mean and unit-variance before inference.

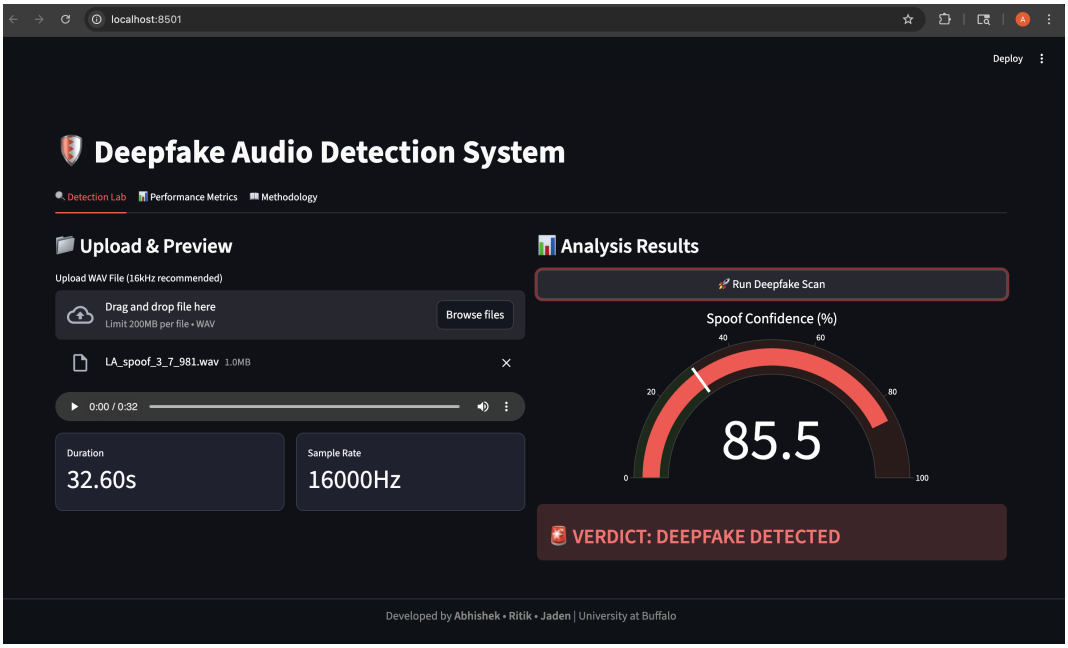

Figure 2: User Interface of the Deepfake Audio Detection System depicting a positive spoof detection with 85.5% confidence.

To handle variable-length audio, the analysis implements a **Dynamic Batch Windowing** strategy. The audio is segmented into 10-second (160,000 samples) overlapping windows. For longer recordings, the script limits the processing to a maximum of five strategic segments to maintain computational efficiency while ensuring broad coverage of the clip. The final "Spoof Confidence" is determined by the maximum probability across all processed windows:

$$P_{final} = \max(\sigma(logits_1), \sigma(logits_2), \ldots, \sigma(logits_n)) \tag{2}$$

This maximum-probability approach ensures that if even a small portion of the background audio contains synthetic artifacts, the system flags the entire file as a potential deepfake.

7.0.2 Visualizing Model Calibration

The analysis output features a custom **Plotly Gauge Chart** that visualizes the model's certainty. This gauge is calibrated with a classification threshold of 0.3, a decision made to prioritize high Precision (93.70%) in the LENS-DF domain. As shown in the detection lab interface, the results are color-coded—green for authentic and red for deepfake—allowing for immediate visual verification of the model's "confidence" in its forensic scan.

7.0.3 Automated Metric Reporting

Beyond individual file testing, the analysis script includes a dedicated "Performance Metrics" dashboard. This module provides a high-level summary of the Phase 2 evaluation, specifically reporting the Accuracy (87.00%) and the F1-Score (0.8592). By integrating these metrics directly into the analysis tool, we can verify if the real-time predictions align with the statistical performance observed during the testing phase on the EnvSDD and LENS-DF datasets.

7.0.4 Methodological Interpretability

Finally, the script provides a "Methodology" context that explains the model's focus on **Vocoder Artifacts** and phase shifts. While the detection system does not utilize a vocoder for audio generation, it is specifically

fine-tuned to recognize the trace signatures left by neural vocoders (such as HiFi-GAN or WaveNet) during the speech synthesis process. These artifacts—including phase discontinuities and spectral leakage—are often inaudible to human listeners but are captured by the Transformer's self-attention mechanism, which analyzes the temporal dependencies across the raw waveform.

8 References

ENVSDD: https://arxiv.org/abs/2505.19203
Lens-DF: https://arxiv.org/abs/2507.16220
AASIST: https://arxiv.org/abs/2110.01200
wav2vec: https://arxiv.org/abs/2006.11477
https://librosa.org/doc/latest/generated/librosa.feature.melspectrogram.html